

al Review

- Compile results, document findings, and conduct stakeholder review
- Provide clear next steps for feature teams

Bridge Process Outputs

A successful Bridge Process delivers:

1. Legal approval status of the model for production use
2. An evaluation report based on our validation dimensions
3. Stakeholder review and a go/no-go decision
4. If approved, an iteration plan with target dates for model deployment

Requesting a Bridge Process Evaluation

Feature teams can initiate a Bridge Process by:

1. Creating a dedicated issue using the Model Validation Rapid Eval Request template
2. Filling out the companion Legal intake issue to initiate the legal review

The AI Framework team reviews all Bridge Process requests within one business day.

Validation Report Structure

For each evaluated model, we generate a comprehensive validation report containing:

Performance Assessment

Comparative latency analysis against current production models:

- Response time benchmarks for standard input sizes
- Tail latency comparison (p95, p99)
- Time-to-first-token performance

Resource requirements to match current performance baselines:

- GPU/CPU configurations needed
- Memory footprint comparison
- Infrastructure scaling requirements

Cost Evaluation:

- Relative cost positioning using a simplified three-tier framework:
 - Comparable to current baseline
 - Moderately higher cost impact (2-5x baseline)
 - Significant cost differential (>5x baseline)

Partnership Status:

- Current relationship status with the model provider
- Impact on decision-making when relevant

Legal Status

Legal team's formal approval status (Approved/Not Approved)
Any conditional requirements or restrictions if approval is granted

Quality Evaluation Results

Our quality assessment examines both raw model performance and feature-level implementation:

Short-term Evaluation Metrics

Initial quality assessment focuses on GitLab's core evaluation suite, measuring:

- Duo Chat performance metrics (readability, conciseness, correctness)
- Code Suggestions accuracy
- RCA averaged correctness scores
- Vulnerability Resolution correctness scores

Long-term Monitoring Framework

We establish ongoing monitoring for:

- Infrastructure stability (Apdex scores)
- Workflow and tool response reliability
- Upstream service availability
- Feature-specific quality metrics

Ownership and Responsibilities

Model Validation Team Responsibilities

- Technical validation (resource usage, scalability)
- Market monitoring and model pipeline management
- Process coordination

Feature Team Responsibilities

- Dataset creation for feature-specific evaluation
- Feature quality ownership and implementation decisions

Shared Responsibilities

- Model selection and prioritization
- Evaluation criteria development
- Results review and decision making

Related Resources

- AI Model Guidance
- GitLab Model Vendor Selection Policy

SECTION: engineering/ai/data-science/ai-powered/model-validation/projects/_index.md

SECTION: engineering/ai/data-science/ai-powered/model-validation/projects/unreview.md

Introducing UnReview

Overview

Code review is an important part of any development process. Over the years, code review has shown a large impact on improving the quality of source code. In addition, code review facilitates in transferring knowledge about the codebase, approaches, expectations regarding quality, etc.; both to the reviewers and to the author.

In a small project, finding an appropriate code reviewer is not a big issue. This might be done manually. However, as the project grows, finding an appropriate code reviewer becomes increasingly difficult. In this setting, manual assignments become time-consuming and require keeping in mind lots of aspects when looking for the right person. Random assignments will be potentially error-prone, as the pool of developers and possible reviewers usually grows with the size of the project, reducing the likelihood of appropriate recommendations. The other option of always assigning key reviewers would make the selected persons overburdened and a bottleneck due to inadvertently siloing knowledge. Overall, none of the provided strategies appear to be optimum. To find appropriate code reviewers in the most effective way, the UnReview project has been initiated.

UnReview focuses on achieving the following goals:

- Eliminating the time wasted manually searching for an appropriate code reviewer to review code changes.

- Making optimum recommendations that consider the reviewers' experience and optimize the review load across the team, which additionally facilitates knowledge sharing.

- Providing analytics on the state of code review in the project, explaining why a particular code reviewer is recommended.

Today, UnReview is an early-stage technology. However, significant testing and validation has been done on production data. After the acquisition is complete, we continue to work on the approach, integrating UnReview into GitLab via iteration.

How UnReview works

To make recommendations, UnReview considers the reviewer's experience in the part of the source code proposed by a merge request. For a given project, UnReview automatically collects the commit history and merge requests in order to identify who is responsible for reviewing and for which part of the source code. Using that information, UnReview then trains the model that is able to make appropriate recommendations.

UnReview is able to resolve the cold start problem, i.e., when the proposed source code is

unknown to the recommendation engine. When making recommendations, UnReview additionally tries to balance the review load across the team. Future versions will also consider the context of the merge request, i.e., which source code has been exactly changed and how it affects other parts of the project.

Challenges

Working on UnReview, we have faced the following challenges:

It can be difficult to identify who is an appropriate reviewer for the merge request, as many can contribute, but not everyone has permission to merge.

Optimizing the review load while making it clear why the person is selected as an appropriate reviewer.

Previous assignments are potentially error-prone. They can be performed at random or a key reviewer is always assigned. Thus, relying solely on previous assignments may lead to biased recommendations.

Architecture

UnReview consists of multiple components used for a variety of purposes, from data extraction and processing to training machine learning models:

Apache Kafka: distributed event streaming and processing;

Apache Hive: data preprocessing including building train/test datasets;

Azure Data Factory: orchestrating the preprocessing pipelines, managing the Hive cluster, and moving the processed data to MongoDB

MongoDB: document-based database for storing the processed data

Azure Blob Storage: temporary storage for the trained ML models and raw data

The following chart provides more details on how the UnReview components relate to each other:

mermaid
flowchart TD

subgraph Azure

ExtractStage --> |Kafka| TransformStage

TransformStage --> |Azure Data Factory| LoadStage

TransformStage --> |manual start of training| TrainStage

TrainStage --> LoadStage

LoadStage --> UnreviewBackend

subgraph ExtractStage

GitLab --> |API| Extracteur

Git --> |API| Extracteur

end

subgraph TransformStage

AzureBlobStorage[(AzureBlobStorage)]

AzureBlobStorage --> |Hive, Azure Data Factory| AzureBlobStorage

end

```

subgraph LoadStage
  Mongo[(ProcessedData/MongoDB)]
  Models[(TrainedModels/AzureBlobStorage)]
end

subgraph TrainStage
  ModelTraining <--> HyperparameterTuning --> BestModelSelection
end

subgraph UnreviewBackend
  Artefacts
  Recommender
  Analytics
end
end

```

When integrating UnReview into GitLab, some components can be replaced as we progress through the defined milestones.

Integration into GitLab

The backend work for integration will be primarily handled by the AI Assisted team with help from the infrastructure team. The frontend work will be by both the AI Assisted team and the Create::code review team (PM Kai Armstrong and EM André Luís).

Milestone 1 Reviewer/maintainer assignment architectural plan and PoC

Milestone 2 Customer facing MVC of integration of unreview

Milestone 3 Extend MVC and create plan for unreview functionality for self-managed customers

Milestone 1

Overview

Milestone 1 focuses on creating an UnReview proof-of-concept that works like Reviewer Roulette based on the GitLab product code. This milestone retains the existing UnReview functionality but requires a number of changes to the architectural components of the approach.

The following tasks have to be completed:

Move Azure and GitHub-hosted aspects of UnReview to GCP.

Implement UnReview code and dependent components in the GitLab managed GCP instance.

Replace the small number of libraries with undesirable open-source licenses.

Demonstrate UnReview on the GitLab codebase and compare/contrast with existing Reviewer-Roulette.

More information on Milestone 1 can be found by following its epic.

Architecture

At this milestone, a set of UnReview components has to be replaced according to the objectives specified above:

Apache Hive and Azure Data Factory will be replaced with Apache Beam backend by Google Dataflow for data preprocessing.

Azure Blob Storage will be replaced with Google Cloud Storage/BigQuery for storing the trained ML models and raw data.

MongoDB will be replaced with PostgreSQL for storing the processed data.

The following chart provides more details on how the UnReview components, including the replaced ones, relate to each other:

mermaid
flowchart TD

subgraph GCP

ExtractStage --> |Kafka| TransformStage

TransformStage --> |Kafka| LoadStage

TransformStage --> |manual start of training| TrainStage

TrainStage --> LoadStage

LoadStage --> UnreviewBackend

subgraph ExtractStage

GitLab --> |API| Extracteur

Git --> |API| Extracteur

end

subgraph TransformStage

GCS[(GCS/BigQuery)]

GCS --> |DataFlow| GCS

end

subgraph LoadStage

Postgres[(ProcessedData/Postgres)]

Models[(TrainedModels/GCS)]

end

subgraph TrainStage

ModelTraining <--> HyperparameterTuning --> BestModelSelection

end

subgraph UnreviewBackend[UnReviewBackend]

Artefacts

Recommender

Analytics

end

end

Milestone 2

Overview

Milestone 2 focuses on providing the UnReview functionality to GitLab.com customers and for dogfooding at GitLab. At this milestone, UnReview should be able to connect to a provided project, automatically extract/process data, and periodically retrain the ML models to improve code reviewer recommendations. GitLab.com customers who license and enable this feature will start seeing and experiencing value from the functionality, including the ability to intelligently assign code reviewers to merge requests based on ML models.

To support GitLab.com customers:

Code reviewer recommendations should be implemented behind a feature flag, which will be disabled by default.

Integrate the UnReview UI into the GitLab product following the MVC pattern.

Make it possible to compare UnReview with existing methods by providing recommendations in a comment.

Add the option to automatically assign reviewers recommended by UnReview.

To support dogfooding at GitLab:

Start the dogfooding process.

Make UnReview available to replace or exist alongside Reviewer-Roulette used by GitLab team members for GitLab projects.

Automatically assign reviewers recommended by UnReview.

More information on Milestone 2 can be found by following its epic.

Architecture

At this milestone, UnReview integrates into GitLab without using the public API because it is believed the API will not be fast enough and would possibly put too high of a load on the GitLab infrastructure. The requested data is passed directly to the UnReview infrastructure via Apache Kafka. In this way, Kafka acts as a bridge between UnReview and GitLab, separating the two infrastructures, while allowing UnReview to access the data it needs that is not available through the public API. To solve another issue of periodically retraining ML models, UnReview could potentially use a scheduler, e.g., Apache Airflow.

The following chart provides more details on how UnReview communicates with the GitLab infrastructure via Apache Kafka:

mermaid

flowchart TD

subgraph GCP

GitlabPostgres[(GitLab/Postgress)] --> |Kafka| TransformStage

GitlabPostgres[(GitLab/Postgress)] --> GitlabApp[GitLabApp]

TransformStage --> |Kafka| LoadStage

TransformStage --> |Scheduler| TrainStage

TrainStage --> LoadStage

LoadStage --> UnreviewBackend

```
UnreviewBackend --> UnreviewUI[UnReviewUI]
Recommender <--> GitlabApp
Analytics --> GitlabApp
```

```
subgraph TransformStage
  GCS[(GCS/BigQuery)]
  GCS --> |DataFlow| GCS
end
```

```
subgraph LoadStage
  Postgres[(ProcessedData/Postgres)]
  Models[(TrainedModels/GCS)]
end
```

```
subgraph TrainStage
  ModelTraining <--> HyperparameterTuning --> BestModelSelection
end
```

```
subgraph UnreviewBackend[UnReviewBackend]
  Artefacts
  Recommender
  Analytics
end
end
```

Milestone 3

Overview

Milestone 3 focuses on further improving the UnReview functionality for GitLab.com customers, as well as planning to provide the functionality to self-managed customers. At this milestone, GitLab.com customers will continue seeing and experiencing value from the functionality, including the ability to intelligently assign code reviewers to merge requests based on ML models. Primary feedback from GitLab.com customers and dogfooding at GitLab should be considered and integrated into the product.

To support GitLab.com customers:

- Make the feature flag enabled by default and available to customers GA via configuration options in the GitLab user interface.

- Automatically assign reviewers when the feature flag is enabled.

To support self-managed customers:

- Determine self-managed performance concerns as it relates to performance and storage requirements in the customer-hosted environment and at customers that intentionally have no internet connectivity.

More information on Milestone 3 can be found by following its epic.

Architecture

Delivering the full UnReview functionality to self-managed customers may be challenging. Processing data and training ML models requires significant hardware power and administration efforts from the customer's team itself. We continue to explore various approaches to addressing this issue. Additionally, we are looking towards privacy-preserving machine learning through data obfuscation.

Overall, one way to support self-managed customers might be to:

- securely access only the necessary data stored on the self-managed GitLab infrastructure;
- process the obtained data on the GitLab.com side and then train the ML model;
- send the trained ML model back to the self-managed GitLab infrastructure to serve it on-premise.

The following diagram provides more details on how UnReview may interact with the self-managed GitLab infrastructure:

mermaid
flowchart TD

subgraph Approach

Gitlaborg[GitLab self-managed] --> |1. send the required data|Gitlabcom[GitLab.com]

Gitlabcom --> |2. train the model|Unreview[UnReview]

Unreview --> |3. return the trained model|Gitlabcom

Gitlabcom --> |4. return the trained model|Gitlaborg

Gitlaborg --> |5. serve the model on-premise|Gitlaborg

end

SECTION: engineering/ai/data-science/ai-powered/workflow-catalog/_index.md

Vision

The Workflow Catalog group is focused on developing Workflow Catalog, a catalog of AI Agents, tools, and workflows that can be created, curated, and shared across organizations, groups, and projects.

Team members

Engineering Manager & Engineers

{{< team-by-manager-slug "sam-beckham" >}}

How to reach us

Depending on the context here are the most appropriate ways to reach out to the Workflow Catalog group:

Slack Channel: gworkflowcatalog

GitLab group @gitlab-org/ai-powered/workflow-catalog/engineering (just engineers)

How we work

We're just getting started and will be defining how we work as we settle in to the new team. Here are some links to get us started:

Root Epic: For grouping all the work and setting out a roadmap

Issue board: For all in-flight issues

Team tasks: For all non-product related team issues

Async updates

Frontend-Backend Collaboration

We aim to foster high levels of collaboration between frontend and backend engineers to ensure development velocity and code quality.

Schema-first development: Before implementation begins, frontend and backend engineers collaborate

to design a GraphQL API schema based on UI requirements, user experience needs, and performance considerations.

Parallel development processes: Once the schema is agreed upon, the frontend can proceed using mock data, mock endpoints, or API stubs that match the agreed schema. The backend can focus on implementing the data model, business logic, and actual API schema.

Maintaining alignment: We value great communication. When requirements or schema need to change, we communicate

early through the relevant GitLab issue or in gworkflowcatalog

so our frontend or backend counterparts stay informed of all changes and can provide feedback early to avoid late-stage blockers.

Our tech stack

GraphQL backend and

frontend. All new schema items must be

marked experimental

to let us making breaking changes when we need.

GraphQL subscriptions rather than polling.

Team meetings

Workflow Catalog: Group meeting

Time: Every Tuesday at 05:30 UTC and 10:00 UTC. It's held twice in one day to allow APAC and EMEA to attend.

Purpose: This meeting serves as a general sync meeting to bring up any current issues and blockers.

Agenda: Google Doc (internal only)

Recordings: Google Drive (internal only)

Capacity Planning

We use a lightweight system of issue weighting to help with capacity planning. These weights help us ensure that the amount of scheduled work in a cycle is reasonable, both for the team as a whole and for each individual. The "weight budget" for a given cycle is determined based on the team's recent output, as well as the upcoming availability of each engineer.

Since things take longer than you think, it's OK if an issue takes longer than the weight indicates. The weights are intended to be used in aggregate, and what takes one person a day might take another person a week, depending on their level of background knowledge about the issue. That's explicitly OK and expected. We should strive to be accurate, but understand that they are estimates! Change the weight if it is not accurate or if the issue becomes harder than originally expected. Leave a comment indicating why the weight was changed and tag your EM so that we can better understand weighting and continue to improve.

Weights

The weights we use are:

Weight	Description
--------	-------------

---	---
-----	-----

1: Trivial	The problem is very well understood, no extra investigation is required, the exact solution is already known and just needs to be implemented, no surprises are expected, and no coordination with other teams or people is required. Examples are documentation updates, simple regressions, and other bugs that have already been investigated and discussed and can be fixed with a few lines of code, or technical debt that we know exactly how to address, but just haven't found time for yet.
------------	--

2: Small	The problem is well understood and a solution is outlined, but a little bit of extra investigation will probably still be required to realize the solution. Few surprises are expected, if any, and no coordination with other teams or people is required. Examples are simple features, like a new API endpoint to expose existing data or functionality, or regular bugs or performance issues where some investigation has already taken place.
----------	--

3: Medium	Features that are well understood and relatively straightforward. A solution will be outlined, and most edge cases will be considered, but some extra investigation will be required to realize the solution. Some surprises are expected, and coordination with other teams or people may be required. Bugs that are relatively poorly understood and may not yet have a suggested solution. Significant investigation will definitely be required, but the expectation is that once the problem is found, a solution should be relatively straightforward. Examples are regular features, potentially with a backend and frontend component, or most bugs or performance issues.
-----------	--

5: Large	Features that are well understood, but known to be hard. A solution will be outlined, and major edge cases will be considered, but extra investigation will definitely be required to realize the solution. Many surprises are expected, and coordination with other teams or people is likely required. Bugs that are very poorly understood, and will not have a suggested solution. Significant investigation will be required, and once the problem is found, a solution may not be straightforward. Examples are large features with a backend and frontend component, or bugs or performance issues that have seen some initial investigation
----------	---

but have not yet been reproduced or otherwise "figured out". |

Anything larger than 5 should be broken down if possible.

Weights should account for both development and review time.

Security issues are typically weighted one level higher than they would normally appear from the table above. This is to account for the extra rigor of the patch release process.

In particular, the fix usually needs more-careful consideration, and must also be backported across several releases.

Communication

The Workflow Catalog Team communicates based on the following guidelines:

Always prefer async communication over sync meetings.

Don't shy away from arranging a sync call when async is proving inefficient, however always record it to share with team members.

By default communicate in the open.

Prefer public channels (gworkflowcatalog) over private message for work-related Slack messaging.

SECTION: engineering/ai/data-science/modelops/_index.md

Vision

Build a diverse and global development team to support and drive results across the stage, while maintaining our values and unique way of working.

Mission

Drive results through iterative development as we add ModelOps teams to the stage, and feature to the product.

Our teams are data-driven, support dogfooding, and collaboration within GitLab and the wider community.

Teams

- MLOps - building on prior work from within the stage.
- DataOps - currently not staffed, a future team to focus on supporting data workloads that power modern software applications built with GitLab.

How to contact us

- Tag a team member in a merge request or issue
- Post a message in the smodelops Slack channel (internal only)

SECTION: engineering/ai/data-science/modelops/mlops/_index.md

Vision

Our product vision is to augment GitLab such that Data Scientists and ML Engineers work together in GitLab to build, experiment, deploy, monitor and keep models secure and up-to-date. Their processes are governable, reproducible, automated, collaborative, scalable, and monitored.

Team

Mission

We exist to bring Data Scientists and ML Engineers into GitLab to collaborate and contribute!

There is a huge market opportunity, as more and more companies and organizations lean into building machine learning models to power their decision making and to integrate the models directly into their products. For many, data science workflows had previously existed in black box silos, we are building the tools that enable the data science teams to join the rest of the org on GitLab to collaborate, build, test, and bring security and governance to the entire software stack.

Team members

The following people are members of the MLOps Group:

Who	Role
Sean Carroll	Engineering Manager
Alper Akgun	Staff Fullstack Engineer
Kevin Chu	Product Manager
Fred de Gier	Staff Backend Engineer

How to contact us

- Tag a team member in a merge request or issue
- Post a message in the gmlops Slack channel (internal only)

How we work together

We are a small team and as we ramp up in a greenfield space favor tight connections and coordinations over rigid prescribed processes.

To keep others informed, we aim to:

1. Publish our plans for the upcoming milestone
1. Update our ever-green MLOps Direction page
1. Include any shipped features in the GitLab Release Post

SECTION: engineering/ai/editor-extensions/_index.md

The Create:Editor Extensions Group is responsible for all aspects of the product categories that fall under the Editor Extensions group of the Create stage of the DevOps lifecycle.

Group overview

Group members

```
{{< team-by-departments departments="Create:Editor Extensions FE Team,Create:Editor Extensions BE Team" >}}
```

Stable counterparts

The following people are stable counterparts of the Create:Editor Extensions Group:

```
{{< group-by-slugs dashaadu amyqualls jaymccure tvanderhelm truegreg>}}
```

Product categories

The Editor Extensions group is responsible for the following product categories:

- Editor Extensions

Communication

We have a sync meeting as a team once per week.

Recordings are uploaded to the Editor Extensions Category playlist on GitLab Unfiltered.

When the discussion is finished, we stop the recording and stay on the call for the remaining time to have a group coffee chat.

Additionally to our main team's slack channels, each extension/project we work on has its own slack channel:

- Team:

- geditor-extensions - Sync Agenda
- geditor-extensionssocial
- geditor-extensions-standup

- Projects

- Language Server: flanguageserver
- VS Code extension: fvscodextension
- Visual Studio extension: fvisualstudioextension
- JetBrains extension: fjetbrainsplugin
- Neovim extension: fneovimplugin

Shared calendars

- Editor Extensions Shared Calendar (Calendar ID:

Cross-group ownership and boundaries

Editor extensions systems host features and modules owned by different groups.

The Ownership and Boundaries page provides clarity and a clear expectation between all parties who author/maintain features in our systems.

Group processes

Our group processes are documented in this section.

If a process is in use but not described here, please follow the guidance in Evolving the process to document it.

Our group is relatively new, and currently light on processes.

There are several differences between how we operate and the shared processes documented in the product development flow and engineering workflow.

Evolving the process

We take an iterative approach to our processes.

Instead of defining it all at once, we add process when we see a gap or identify a problem. For efficiency, we also don't shy away from removing a process that has outlived its usefulness.

To evolve the process, please start with an MR to this page which contains your specific proposal, and ask for feedback.

If an issue is better suited for the discussion, it should be created in the meta project.

Epics & Issues

We exclusively use issue/epic descriptions as the single source of truth for our planned work.

- Epics: GitLab Epic Search
- Issues: GitLab Issue Search

Epics and issues are created in the project that matches their scope in the narrowest possible way. We use the following projects:

- Work specific to a single extension or to the Language Server:
 - Language Server
 - Visual Studio Extension
 - VS Code Extension
 - JetBrains Extension
 - Neovim Extension
- Other projects:
 - Editor Extensions Meta - Extensions work that isn't specific to a single extension, and general team-related issues
 - Code Suggestions - Code Suggestions work that isn't specific to Extensions

- GitLab - Monolith work

Prioritizing

We use Milestone Planning Issues to define our goals for the current/upcoming milestone. The PM and EM are responsible for aligning on the goals. The planning issues are created automatically every month.

We use the Editor Extensions Priority Board to track the relative priority of issues. Issues at the top of a column have the highest priority.

Separately, the technical writer for this group also triages open issues for potential documentation and UI text changes, and follows the Technical Writing triage process. After review, each issue receives the ~tw::triaged label.

Technical debt

We mark technical debt with the ~tech-debt label.

Language Server + VS Code Extension

We prioritise technical debt in the monthly TypeScript Contributors meeting (example meeting issue). This meeting can be found in the team calendar (also in the frontend and Create stage calendars).

The detailed process for setting up the meeting can be found in this team snippet.

We vote on all technical debt issues with emoji reactions and then discuss the top priority issues in the meeting. The most upvoted issues are prioritised (if capacity allows it) in our next milestone.

Estimates

We use three weights to give a rough estimate of the issue's complexity. The weight is made out of two parts:

Base weight

These are estimates for someone familiar with the codebase/system.

- 1 - day or two of effort
- 2 - week of effort
- 3 - week and a half of effort

Everything with a base weight above 3 should be a spike that will result in one or more issues

with estimated weight.

Subjective weight

You can add extra weight 1 or 2 to the base weight if you are new to the team/codebase/system.

Examples:

- Bob is new to the team, they are picking up an issue with weight 1 (i.e. simple). They know the technologies, but not our system. They'll add 2 subjective weight making the final weight 3.
- Alice is familiar with the system but is picking up an issue (weight 2) that touches authentication. She's unfamiliar with our authentication patterns and adds 1 subjective weight to have enough time to review the existing approaches. That makes the final weight 3.

Development Workflow

The work in progress is captured on our Workflow Board.

For issues to appear on this board, they must have ~"group::editor extensions" label and current milestone.

We use the following subset of the workflow labels to indicate the state of the issue:

- ~"workflow::ready for development" - the issue has been described, estimated and scheduled, and it's ready to be picked up and worked on.
- ~"workflow::in dev" - the issue has an assigned person who started implementing it.
- ~"workflow::in review" - the spike/implementation is finished, and someone needs to review the spike result or the last MR (the last MR because if there are more MRs to implement, only the last one should result in the change of the workflow label).

Code review

Comment severity levels

We are using the following levels of comments in our MR reviews. The MR reviewer decides on the severity of their comments and discusses this judgement with the MR author.

Immediate follow-up and follow-up levels have more complex processes attached to them. You can link to this page in your reviews to explain how the follow-ups should be tracked and addressed.

- blocking - (blocking) - Must be addressed in the same MR. All comments without any decorator are blocking, so adding the (blocking) decorator is optional.
- immediate follow-up - (non-blocking, immediate follow-up) - The comment is important enough that the fix should be implemented as the very next MR the author creates (emergencies like critical production bugs still take priority over the follow-up).
- follow-up - (non-blocking, follow-up) - This issue should be addressed but the severity should be weighed against all other technical debt and follow-ups
- optional - (non-blocking) - this comment is either personal preference or doesn't have a large impact on the MR. Creating a follow-up issue is optional.

When to use immediate follow-up

These are the reasons why we would choose immediate follow-up over a blocking comment:

- The original MR was too complex and we are trying to re-focus - The MR review has many comments and it has been going on for many days, then working on an immediate follow-up might reduce the cognitive load and improve focus.
- The original MR is urgent - The MR delivers an urgent/critical feature, the feature works but the code needs to be still changed to match our standards. If it wasn't for the urgency, we would prefer to do the work in the same MR.

Immediate follow-up process

When an MR with an unaddressed immediate follow-up comment is merged:

- MR author creates an issue for this comment and ensures the issue:
 - Has the MR author is assigned to it
 - Has title and description that captures the work
 - Is marked as blocking issue associated with the original MR
 - Is labeled ~immediate and ~follow-up
- Until the immediate follow-up is implemented, the GitLab issue for the original MR should not be closed.

Follow-up process

1. The MR author creates a follow-up issue and clearly documents (in title and description) what the follow-up is about and adds the ~follow-up label.
1. The MR author references the issue in a // FIXME comment in the code.

Comment Example

> "(immediate follow-up): This introduces a singleton and we are trying to move away from singletons in the whole system (issue link). We should address this quickly to prevent the anti-pattern from spreading."

These comment severity levels are not static. For example, you might initially mark a comment as blocking, but after discussion with the MR author, you may agree to change it to a follow-up.

Tracking issues in code with FIXME comments

We track existing issues directly in the code using // FIXME comments. These comments are created in two ways:

- An engineer notices a pre-existing issue during development that they can't fix immediately. They add a // FIXME comment to the codebase and optionally reference an issue.
- A Follow-up comment has been created during an MR review, and we track this follow-up with both an issue and a // FIXME comment.

Maintainers are allowed to add // FIXME comments as the last commit to the MR before merging it to avoid an extra review cycle.

We use FIXME and TODO comments interchangeably.

For a more detailed explanation and list of advantages of this approach, see the proposal.

Temporary silos

Our team owns several projects, written in different languages (Typescript, Kotlin, C, Lua) and targeting different platforms.

We are currently intentionally working in silos, by assigning engineers to the project/technology that they can contribute to most efficiently.

This increases speed of development as we aim to reach GA for Code Suggestions, at the cost of knowledge sharing and team resiliency.

This intentional siloing is a temporary measure.

While we do need to cultivate deep expertise in each project, the mid-term vision for the team is to provide opportunities for everyone to contribute across each project.

This will increase collaboration, team resiliency, and provide opportunities for growth.

Knowledge sharing

This section contains links to make it easier to find the same information for each extension.

This is a first iteration, this content should probably live somewhere else eventually.

Languages supported by Code Suggestions

Each extension defines an array of supported languages.

- VS Code: `AIASSISTEDCODESUGGESTIONSLANGUAGES`
- Visual Studio: `LanguageManager`
- JetBrains: `SUPPORTEDEXTENSIONS`
- Neovim: `autofiletypes`

Language Server versions in use

- Visual Studio: the server binary is versioned directly: `GitLab.Extension/Resources/gitlab-lsp-win-x64.exe`
- VS Code: the server is pulled as a package: `package.json`
- JetBrains: N/A - see issue
- Neovim: the server is pulled as a package: `lsp/package.json`

Hiring

Check out our jobs page for current openings.

Engineering metrics

```

{{< tableau height="600px" toolbar="hidden" src="https://us-west-
2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard
">}}
  {{< tableau/filters "GROUPLABEL"="editor extensions">}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-
west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1">}}
  {{< tableau/filters "GROUPLABEL"="editor extensions">}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-
west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues">}}
  {{< tableau/filters "GROUPNAME"="editor extensions">}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-
2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard"
>}}
  {{< tableau/filters "GROUPLABEL"="editor extensions">}}
{{< /tableau >}}

```

SECTION: engineering/ai/editor-extensions/ownership.md

Context

Multiple groups contribute to features exposed through our editor extensions. This can create ambiguity regarding ownership and maintenance responsibilities. This document outlines an agreed-upon approach for consistent ownership attribution, aligning expectations and improving efficiency among all involved parties.

Owning a system part means being responsible for feature development, bug-fixes, and general maintenance of the affected part. Ideally, a team has maintainers in the system part they own to be more independent of the editor extensions team, but this is not a strict requirement.

System overview

Currently, the external features being integrated via our Editor Extensions follow this architecture:

UI component built on gitlab-ui

Integration with Editors: Webview

Webview is a web page shown in the editor

Integration with Editors: Business logic

Business logic represents the editor-specific integration (e.g. TypeScript and VS Code Extension API, Kotlin and JetBrains Plugin API)

Legacy Ownership model

Currently used by Duo Chat:

System part	Group responsible
-----	-----
Chat UI component in gitlab-ui	AI-Powered:Duo Chat group.
VS Code - Chat Webview (web page)	AI-Powered:Duo Chat group with support from Create:Editor Extensions group
VS Code - Chat business logic (extension)	Create:Editor Extensions group
JetBrains - Chat Webview (web page)	AI-Powered:Duo Chat group with support from Create:Editor Extensions group
JetBrains - Chat business logic (plugin)	Create:Editor Extensions group

Ownership model for features implemented in Language Server

Currently used by Duo Workflow and soon to be used by Duo Chat.

System part	Group responsible
-----	-----
UI Component	Group owning the UI component
LS webview & plugin	Group owning the UI component with support from Create:Editor Extensions group
VS Code integration	Create:Editor Extensions group
JetBrains integration	Create:Editor Extensions group
Visual Studio integration	Create:Editor Extensions group

Process of notifying others

When a team starts working on a feature that will require Editor Extensions support:

PM/EM will notify Create:Editor Extensions group's PM/EM - this allows the team to forecast the capacity necessary to support in the near future

Feature implementation

Legacy ownership model (Duo Chat implemented separately in all extensions)

When the UI of the feature is ready:

the authoring team opens an integration MR

The Create:Editor Extensions group will assign an engineer to take over the MR and complete integration

Language Server ownership model

When the Language Server change is done, the feature team will notify Create:Editor Extensions group and they'll integrate the language server change to the editor extensions.

SECTION: engineering/ai/search/_index.md

Vision

The Global Search Group focuses on bringing world class search functionality to GitLab.com and self-managed instances.

This page covers processes and information specific to the Global Search group. See also the

Global Search and Code Search direction pages.

Mission

The group is responsible for improving and expanding upon our current global search implementations using Elasticsearch, PostgreSQL, Zoekt, and Gitaly. Areas of responsibility will include global search functionality, UI, ingestion mechanisms, optimal indexing, administrative tools, and installation mechanisms for self-managed installations.

Additionally, we build and maintain critical AI context infrastructure, including:

- AI Context Abstraction Layer: A unified interface for Retrieval Augmented Generation (RAG) across multiple vector databases (Elasticsearch, OpenSearch, PostgreSQL with pgvector), enabling AI features to work regardless of underlying storage
- GitLab Zoekt: GitLab's scalable exact code search service and file-based database system, with flexible architecture supporting various AI context use cases beyond traditional search. It's built on top of open-source code search engine Zoekt.

These systems will be fundamental to providing high-quality context for AI features via Retrieval Augmented Generation work, which includes:

- Identifying and preparing new useful data for our AI-powered features in collaboration with feature teams and the AI Framework team
- Storing vector embeddings of epics, issues, MRs, source code, and more
- Providing retrieval APIs for those vector embeddings, metadata filtering, and ensuring permissions are enforced
- Enabling fast, precise code search and context retrieval essential for AI context

This team doesn't own custom searches for specific features, such as the "filter bar" on issues which is part of the Issue Tracking category owned by the Project Management group.

Team Members

The following team members are permanent members of the Global Search Group:

{{< team-by-manager-slug manager="cliu" role="Global Search|Core Platform" >}}

Stable Counterparts

The following members of other functional teams are our stable counterparts:

Name	Role

Ben Venker	Senior Product Manager, Global Search
Ashraf Khamis	Senior Technical Writer
Cleveland Bledsoe Jr	Senior Support Engineer
Brenda Nyaringita	Support Engineer(EMEA)

Shared Responsibilities

The Global Search team shares responsibilities with the AI Framework team in the area of (RAG). Specifically, we will collaborate in the data preparation stage and information retrieval stage of the RAG process.

AI Context Infrastructure and Advanced Search data stores

The Global Search team maintains several key systems that power both traditional search and AI context capabilities:

Core Infrastructure Components

- Elasticsearch: Powers Advanced Search functionality with full-text search, aggregations, and vector similarity search capabilities
- GitLab Zoekt: GitLab's scalable file-based database system providing exact code search with enterprise-scale performance (48+ TiB indexed on GitLab.com). Beyond code search, Zoekt's flexible architecture serves as a foundation for various AI context use cases
- AI Context Abstraction Layer: A unified Ruby gem interface enabling RAG across multiple vector databases (Elasticsearch, OpenSearch, PostgreSQL with pgvector), ensuring AI features work regardless of underlying storage solution

These systems work together to provide comprehensive search and AI context capabilities, from traditional keyword search to sophisticated vector similarity matching for AI features.

Advanced Search as an Enabling Framework

Beyond powering GitLab's global search functionality, Advanced Search serves as a critical framework that enables other teams across GitLab to overcome the inherent limitations of PostgreSQL for complex search and analytics use cases. Teams leverage Advanced Search infrastructure to:

- Scale beyond PostgreSQL constraints: Handle large-scale text search, aggregations, and analytics that would be prohibitively expensive or slow in PostgreSQL
- Enable sophisticated filtering: Support complex multi-field queries, faceted search, and advanced filtering capabilities
- Power analytics and insights: Generate aggregations, statistics, and insights from large datasets without impacting primary database performance
- Support AI and ML workflows: Provide vector similarity search, embeddings storage, and retrieval capabilities essential for AI features

This framework approach allows feature teams to focus on their domain expertise while leveraging battle-tested, scalable search infrastructure maintained by the Global Search team.

A note on basic search

Basic search utilizes Postgres for text searching and Gitaly for code searching. Both functionalities are significantly limited compared to with Advanced search.

Current state of Advanced Search scopes

There are many data types and search scopes already available via the Advanced Search interfaces. Below is a table that outlines the various available data types and the status of various functional elements, such as permissions, cross-group searching, and embeddings.

Data type / scope	Privacy / Permissions	Cross-namespace / cross-group searching	Keyword search	Similarity search & Embeddings	Metadata filtering
-----	-----	-----	-----	-----	-----
Code	Yes	Yes	Yes	In progress	Group, Project, Include/exclude archived, include/exclude forks, Language, Filename, Path, Extension
Issues	Yes	Yes	Yes	Yes	Group, Project, Status, Confidentiality, Labels, Include/exclude archived
Merge requests	Yes	Yes	Yes	No	Group, Project, Status, Include/exclude archived
Epics	Yes	Yes	Yes	No	Group, Project
Comments	Yes	Yes	Yes	No	Group, Project, Include/exclude archived
Users	Yes	Yes	Yes	No	Group, Project
Commits	Yes	Yes	Yes	No	Include/exclude archived
Milestones	Yes	Yes	Yes	No	Group, Project, Include/exclude archived
Project	Yes	Yes	Yes	No	Group
Wiki	Yes	Yes	Yes	No	Group, Project

Meetings

Whenever possible, we prefer to communicate asynchronously using issues, merge requests, and Slack. However, face-to-face meetings are useful for establishing a personal connection and addressing items that would be more efficiently discussed synchronously, such as blockers.

- The Global Search Group meets weekly on Tuesdays at 14:00 UTC.
- The Global Search Group also has an Open Discussion Hour on Thursdays at 12:30 UTC

Work

We follow the general workflow and principles defined in Product Development Flow and Engineering Workflow. To bring an issue to our attention, please create an issue in the relevant project. Add the ~"group::global search" label and any other suitable labels. If it is an urgent issue, please reach out to the Product Manager or Engineering Manager listed in the Stable Counterparts section above.

Below are a few guidelines the team follows in the day-to-day work.

- We use asynchronous communication with each other and with other GitLab teams via GitLab, Slack, Google Docs, etc.
- We have weekly team meetings, 1-on-1 meetings, and virtual happy hours via Zoom to discuss various topics and create team bonding.
- We encourage all backend engineers in our team to have their changes reviewed by someone else in our group. It's great for knowledge sharing.
- We organize our tasks under Epics and Issues. The Product Manager and Engineering Manager go through the backlog at the planning phase of each release and put issues into the next one or two milestones. The issues on the milestone board are sorted based on priority. The higher priority issues are placed on the top.

- We apply the Deliverable label to the issues that we intend to close in a milestone before the milestone starts. Issues added during a milestone should not have the Deliverable label applied. We review these issues in the middle of the milestone, usually the first week of each month. We will remove the Deliverable label from the issues that are not likely to make it into the release.
- We apply the Stretch label to the issues that we intend to start during a milestone but are not committing to closing.
- We work with the UX team for features that need their design input by labeling the issues with a UX workflow label and adding the corresponding UX team counterpart as the assignee. We use workflow::problem validation and workflow::solution validation for user research and workflow::design for UI design and prototyping. Once the design is finished, workflow::ready for development label will be added as an indicator that development can start. For minor UX/UI changes, we contact our UX counterpart or the Product Design Manager to request a review for fast iterations.
- We work with the Quality team for issues that require input from a testing perspective by labeling the issues with workflow::planning breakdown and adding the SET counterpart as an assignee. Once SET reviews the issue, they acknowledge back with the label quad-planning::complete-action or quad-planning::complete-no-action
- We work with the Technical Writing team for issues that need documentation change by labeling the issues with `documentation` and adding our counterpart in the Technical Writing team as assignee. Our technical writer helps us update the corresponding document. The documentation change normally happens together with the code change.
- We work with our stable counterpart in the Security team for issues that need input from a security perspective. We suggest using team planning issues, for example, this one, for communication.
- We work with the Support Engineering team by collaborating on issues directly. We invite our counterpart in the Support Engineering team to our team meeting every month to have direct communication.
- When team members are ready for their next tasks, they will pick an issue from the milestone board and become the issue owner by assigning the issue to themselves. Team members should prioritize issues with the Deliverable label. The issue owner will be responsible for finding the solution to the issue. They can propose a solution by opening a Merge Request. They can also break down the issue into smaller sub-issues if it makes sense to take an iterative approach.
- Before going out of office for an extended time, assign items still in review to the Engineering Manager. The Engineering Manager can reassign as needed.
- Whenever a team member reviews an author's work that is out of office for an extended time, they are welcome to complete the changes requested if they deem themselves comfortable with the remainder of the work.
- We review and prioritize bugs every week. It is common for bugs to represent the problem without identifying the impact. Because the Product Management and QA share the responsibility of assessing every bug for priority, severity, and details. Severity uses an approximation of the Risk Matrix to identify potential risk and frequency. Priority is based on total impact over time. Occasionally, something of a lower priority/ severity will be added to a milestone when it relates to work currently scheduled.
 1. Review all new bugs for content, priority, severity, and milestones
 1. Review any bugs missing priority or severity
 1. Prioritize bugs for the current milestone. 10% of scheduled work should be focused on bugs
 1. Schedule bugs for future milestones based on capacity, severity, priority, and

relationship to any scheduled work

Breaking changes process

Before a major milestone starts, we prepare an epic with all the breaking change issues linked. As usual, we work to get approvals but keep the MR in draft to prevent it from merging before the major milestone. If an MR is independent, we can have the master as a target branch. If not, we can have a sequence of MRs with target branches set to each other. As soon as the first one merges, the next will automatically target master.

Every MR that was created before the breaking change milestone should have this or a similar warning in the description: :warning: This MR must be kept as a draft and cannot be merged until DATE :warning:

Bugfix backport process

We review the bugfix merge requests every week. To facilitate this process, we have created scoped labels: backport::required, backport::skip, and backport::complete.

- The backport::skip label will be added to merge requests if no backport is needed.
- The backport::required label will be added to the merge requests that need to be backported to a previous release in the initial review. The DRI will follow the patch release process to backport the fix to a previous release. Once the backport is done, the backport::complete label will be added to indicate the whole process is complete.

Advanced Global Search Rollout on GitLab.com

The team has been actively working on enabling Elasticsearch powered Advanced Search on GitLab.com. Based on our analysis, we set our first target to roll this feature out for all the paid groups on GitLab.com. You can find more details about the timeline and progress in the links below.

- Plan of Advanced. Search Rollout on GitLab.com
- Steps and Enhancements of Advanced Search Rollout on GitLab.com

Severity Labels for Search Issues (~advanced search, ~global search)

Type of Operation	~severity::1 - Blocker	~severity::2 - Critical	~severity::3 - Major
~severity::4 - Low			
----- ----- ----- -----			
----- -----			
Recall Record, Global	Above 10 seconds to timing out	Between 7 and 10 seconds	Between 4 and 7 seconds
	Between 2 and 4 seconds		
Time until inserted record is recallable	Above 15 minutes	Between 15 and 10 minutes	
	Between 10 and 5 minutes	Between 3 and 5 minutes	

The two types of operations we detail severity metrics for above are:

- Recall Record, Global: This is the time it takes to recall a record using a globally scoped search of GitLab.com. Records could be entities such as projects, users, groups, etc.

- Time until inserted record is recallable: This is the elapsed time between adding a new record and having that new record be recallable via a search. This process depends on many underlying technologies such as the Go indexer, Sidekiq queues, and the Elasticsearch database.

Weighting for Search Issues

We use the Fibonacci rating system to assign weights to Search issues. Below are a few guidelines when setting issue weight:

- Issues that include ~backend and ~frontend work should have the weights added for a total weight representative of the work effort.
- Spike issues are assigned a weight to help timebox the effort.
- Bugs will not be given a weight.
- Any issue weighted over 5 should be broken down into smaller iterative steps if the issue does not contain ~backend and ~frontend work.

Weight	Description
----- -----	
0	No effort or trivial effort (example: Documentation typo or Feature Flag Rollout)
1	Low effort (No Database migrations or Advanced Search migrations)
2	Low-Medium effort
3	Medium effort
5	High effort

MR reviews

We have the following guidelines for doing reviews on Global Search Team MRs:

- The MR author is responsible for deciding if the initial or maintainer reviews should be done by a Global Search Team member and can indicate that in a comment or by assigning the reviewers.
- Draft status indicates that the MR is not ready to be merged, but the author could decide to assign a reviewer while in draft mode. Unless a review is urgent, the author should wait for the pipeline to pass before assigning a reviewer.
- We use Conventional Comments to communicate effectively in review comments.
- The merge request author resolves only the threads they feel they have fully addressed and all discussions have been closed, anything else is resolved by the reviewer. When a merge request has many threads, it is helpful for the reviewer to go back to open threads to pick up where the previous discussions were left off.

Oncall escalation coverage

As the Global Search Team requires special domain knowledge, such as Elasticsearch, we borrow team members with this domain knowledge from other groups to cover the on-call escalation when we are understaffing, especially during the holiday seasons. In general, we will follow the dev on-call process. The Elasticsearch domain experts, identified by domainexpertise on their profile, may be contacted when SRE and dev on-call engineers cannot resolve the production incidents. We don't expect the domain experts to work outside their normal working hours. In case of emergency, we will follow the rules and best practices outlined in our Incident Management handbook. To assist team members in catching up on the latest development status and

resolving potential incidents, we have created a Global Search Incident Management document as a reference.

Onboard domain experts from other groups to cover production incident escalation

When onboarding domain experts from other groups to help cover production incident escalation, we may consider the following actions:

- Suggest the team member add elasticsearch as their domainexpertise in their team member profile
- Add the team member to the Slack group global-search-team which can be used by SREs and other on-call engineers to contact in case of emergency
- Create the access request for the team member to grant them access permissions to Elasticsearch cluster
- Schedule walk-through sessions with the team member to go over the latest architecture and development status

Offboard domain experts from production incident escalation coverage

- Remove the team member from the Slack group global-search-team
- Revoke the team member's access permission of Elasticsearch cluster

Common Links

- Global Search Team Milestone Board
- Global Search Team Workflow Board
- Global Search Team Epics
- Global Search team slack channel (internal) gglobalsearch
- Global Search Roadmap
- Bug Review Board

JTBD

We utilize the Jobs to be Done (JTBD) framework to better understand our customers' and users' needs. You can view the current list of our JTBD here.

Performance Testing

We are exploring Rally for performance testing the Elasticsearch cluster. Workload data is determined using Kibana and stored in a Google Sheet (internal)

Resources

Documentations

Search and Advanced Search

- GitLab Elasticsearch Integration
- GitLab Advanced Search Development Guidelines
- GDK Elasticsearch Setup Instructions

AI Context Infrastructure

- Zoekt Design Document - Comprehensive architecture and implementation details
- GDK Zoekt Setup Instructions
- AI Context Abstraction Layer Design Document - Unified RAG interface architecture
- AI Context Abstraction Layer Source Code - Ruby gem implementation

Blog Posts

- Lessons from our journey to enable global code search with Elasticsearch on GitLab.com
- Update: The challenge of enabling Elasticsearch on GitLab.com
- Update: Elasticsearch lessons learnt for Advanced Global Search 2020-04-28
- How the Search Team at GitLab Implemented a Risk Map to Direct Automated Testing Efforts
- GitLab's data migration process for Advanced Search

Product Demos

- GitLab search with Custom Search Engines
- Use the GitLab search bar to navigate
- Search suggestions for recently viewed issues and merge requests
- How to search for epics in GitLab
- Elasticsearch Migrating Issues To Separate Index Technical Overview

SECTION: engineering/ai/search/es-rollout-timeline.md

Steps and Enhancements

- 2019-11-05: Search security rapid action started.
 - Advanced Global Search went through 3 rounds of enable-disable in the prior 2 months due to 3 security issues discovered by HackerOne only hours after going online. See 12.3.3, 12.3.5, and 12.4.1/12.3.6.
 - Team decided to take a systematic approach to discover and fix issues.
 - Rapid action weekly agenda.
 - All engineering attention (2 engineers) swarmed on the security rapid action.
- 2019-12-09: Security rapid action completed.
 - All known security issues were fixed.
 - A comprehensive set of test matrices was executed by the AppSec team.
- 2019-12-16:
 - Advanced Global Search was re-enabled for gitlab-org group on GitLab.com..
 - Only GitLab's own gitlab-org was enabled, restoring to the state before the security rapid action.
 - Margin analysis requested by CFO, cost estimation issue created.
- 2020-01-15: Cost analysis completed.
 - Due to holidays, it took about one month to complete initial estimation and margin analysis.

- 2020-01-17: Financial approval completed.
- 2020-01-24: First two customers were scheduled to get Advanced Global Search enabled.
 - Elasticsearch cluster node crashed due to out of memory during initial indexing.
 - The enabling process was stopped and Elasticsearch cluster was brought offline.
 - Advanced Global Search service was completely offline.
- 2020-01-27: A retrospective of the incident was held.
 - Continued troubleshooting with the Infrastructure team.
 - Started engagement with an Elastic support engineer.
- 2020-01-30: Discovered the root cause of the problem with help from Elastic support.
 - Out of memory was caused by the combination of fairly large bulk requests queued for ingestion and small heap.
 - Started the work of scaling indexing jobs by utilizing Elasticsearch Bulk Import API and Redis sorted set.
- 2020-02-04:
 - Re-enabled the GitLab's own gitlab-org and gitlab-com groups.
 - Enabled the first customer.
- 2020-02-09 ~ 2020-03-05:
 - Iterative learning of enabling new customers and monitoring production environment via indexing 7 new groups one-by-one, meanwhile developing tools and playbook for batch indexing.
- 2020-02-29: Merged Add a bulk processor for elasticsearch incremental updates.
 - It processes incremental database updates in batch. It's more efficient and can lower the load on Elasticsearch when it's busy.
 - It uses Redis sorted set, which can deduplicate the indexing jobs. It can also help lower the load on Elasticsearch cluster.
- 2020-03-06:
 - Merged Use less expensive indexoptions, reduced index size by 36.6%.
 - Merged Bulk API related Elasticsearch version compatibility fix.
- 2020-03-11: First attempt of adding new groups in batch succeeded. Another 30 groups enabled. Total enabled groups were 39, including GitLab's own groups.
- 2020-03-12: Routing feature was turned on GitLab.com, which resulted in 5x-6x latency improvement.
- 2020-03-26: Merged Use prefix search instead of ngrams for sha fields, reduced index size by another 12.3%, total index size reduction is 44.4%.
- 2020-04-07: Optimized re-indexing process to achieve zero downtime.
 - Elasticsearch index alias was used in GitLab.com to make re-indexing operation more flexible, efficient, and robust.
 - Re-indexing does not require downtime any more on GitLab.com.
 - Zero downtime re-indexing related work started.

- 2020-04-08: More groups were enabled and the total enabled groups is about 3%.
- 2020-04-09:
 - More groups were enabled and the total enabled groups is about 6%.
 - Started investigating how to speed up initial indexing of groups
 - Increasing the number of Sidekiq workers has been discussed.
 - The impact on other parts of GitLab.com was also evaluated.
- 2020-04-27: More groups were enabled and the total enabled groups is about 9%.
- 2020-04-28: A blog post was published which shared lessons learned from rolling out Advanced Global Search on GitLab.com.
- 2020-04-30: More groups were enabled and the total enabled groups is about 12%.
- 2020-05-08: Started working on moving repository indexing jobs to Redis sorted sets, which would help
 - Boost indexing performance.
 - Isolate incremental index update and initial indexing to separate job queues, accelerating both indexing job types.
- 2020-05-20: More groups were enabled and the total enabled groups is about 16%.
- 2020-05-24: More groups were enabled and the total enabled groups is about 17%.
- 2020-05-27: Merged remove partial word matching from code search, which would help reduce the storage size significantly.
- 2020-06-04: Another re-indexing was done on GitLab.com. The last storage optimization change was applied. It saved another 75% storage over previous optimizations and resulted in a total of 86.1% index size reduction.
-